Ex. No:8
Date: Testing of Requirements

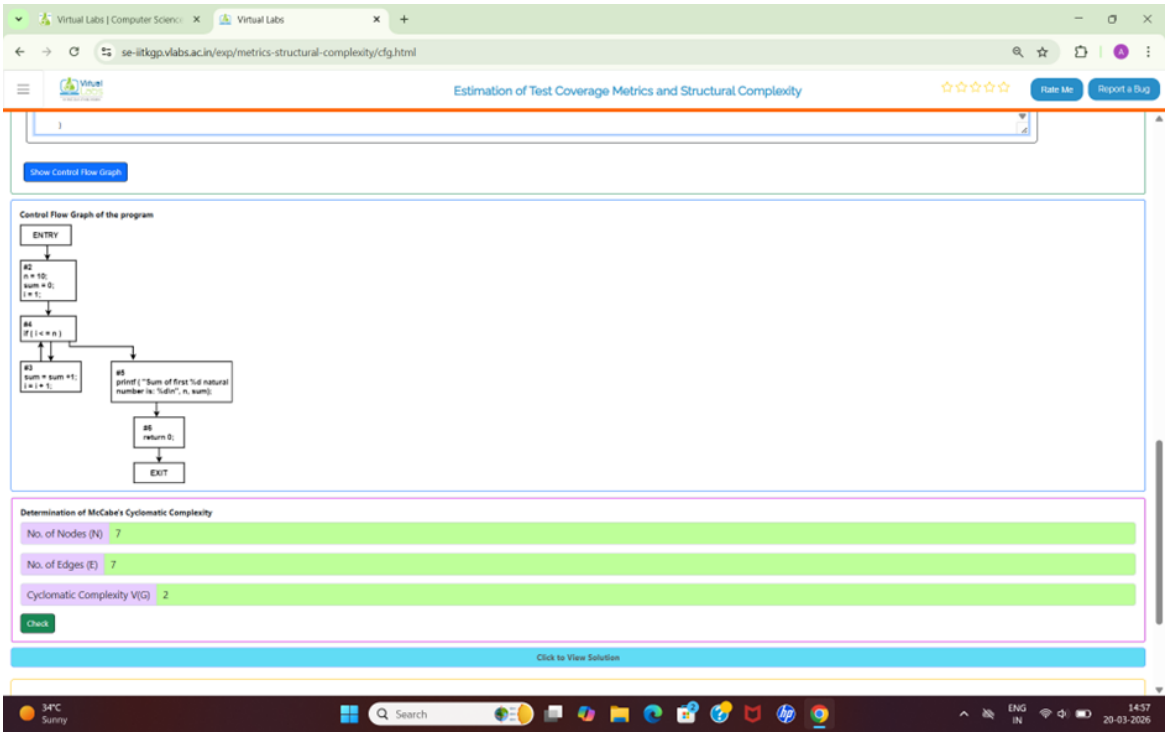
Aim:
The aim of this experiment is to calculate the test coverage metrics and analyze the structural complexity of a given program using its control flow graph (CFG). This includes identifying nodes, edges, and independent paths in the program and computing Cyclomatic Complexity to determine the minimum number of test cases required for proper testing. This experiment also helps in understanding how the structure of a program impacts testing efficiency and overall software quality.

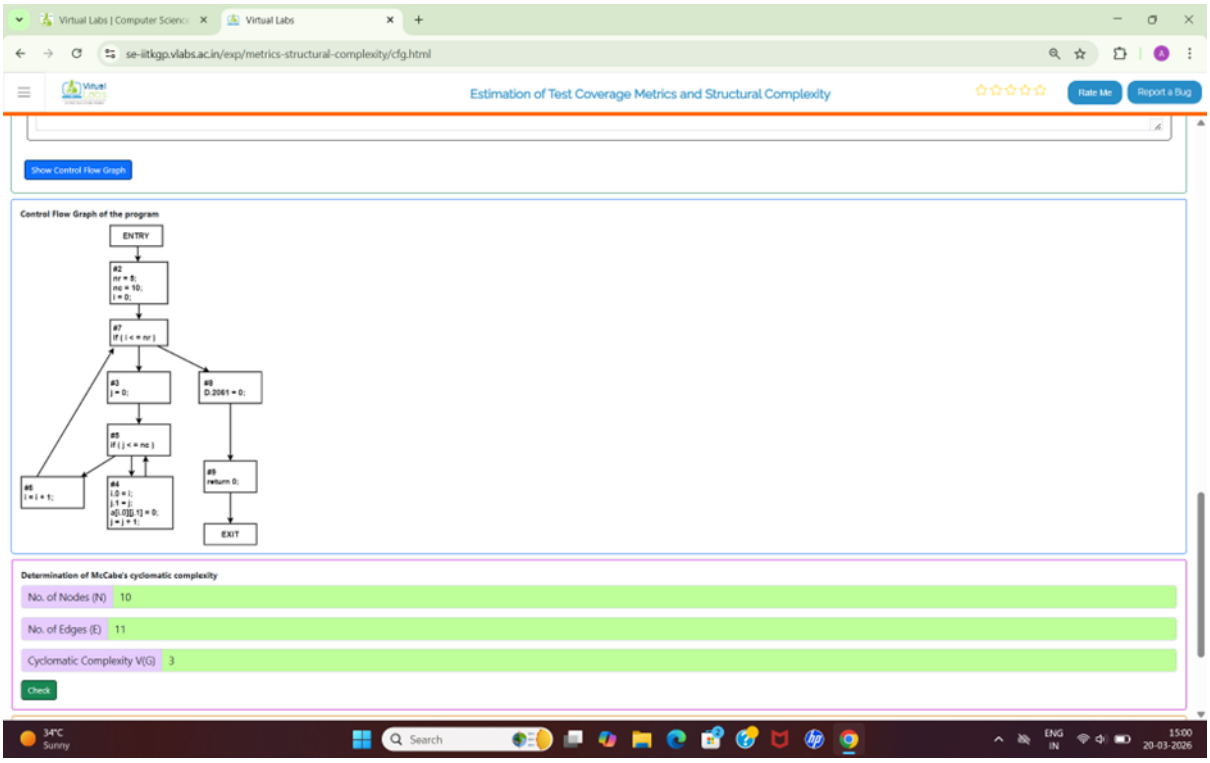
Problem Description:
In software testing, it is necessary to ensure that every part of a program is executed and validated correctly. The given problem focuses on analyzing a program by converting it into a control flow graph, where each statement or block is treated as a node, and the execution flow is represented using edges. By studying this graph, we can evaluate the complexity of the program and determine the number of independent execution paths. This helps testers create test cases that provide maximum coverage and identify logical errors efficiently.

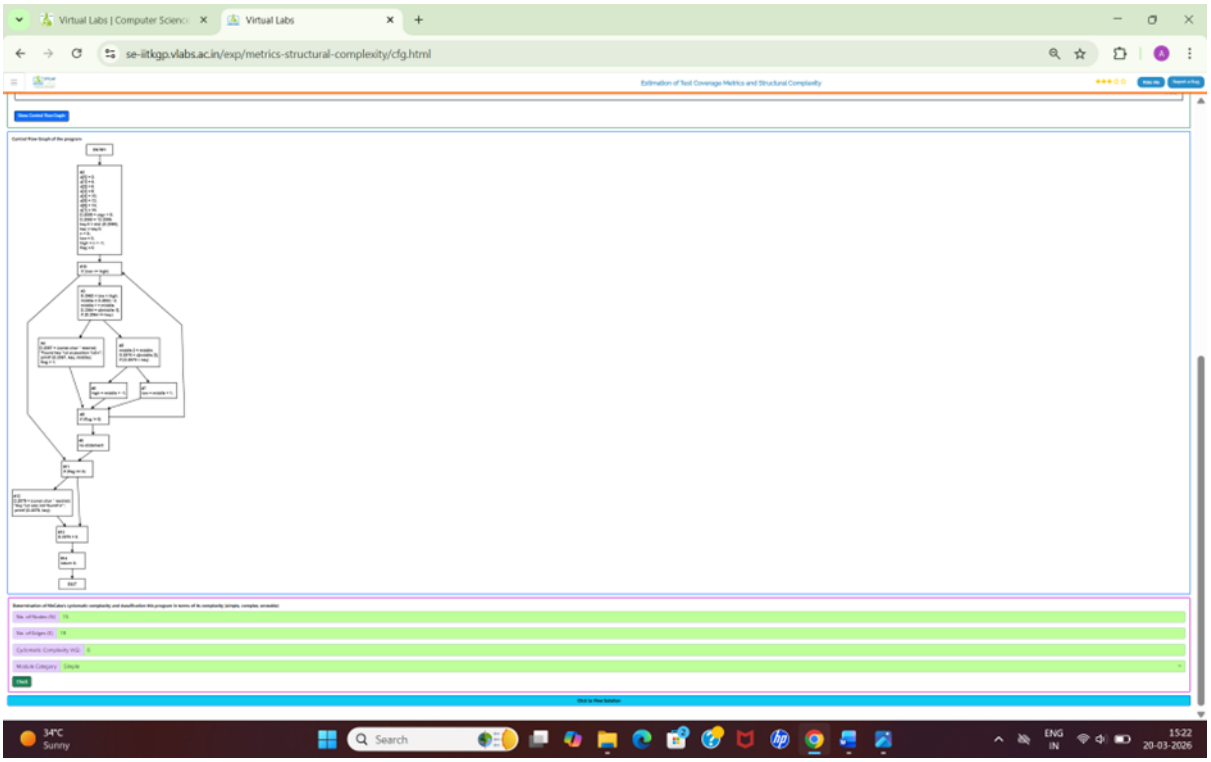
Given Conditions:
The given program includes sequential statements, decision-making constructs (like if/else), and looping structures. The program is assumed to have a single entry and exit point. Each block of code is considered a node, and the control flow between statements is represented as edges. The required values include the number of nodes (N), number of edges (E), and number of predicate nodes (P). These values are used to compute Cyclomatic Complexity and test coverage metrics.

Methodologies:
The methodology followed in this experiment involves first analyzing the given program and dividing it into logical blocks. Each block is represented as a node in the control flow graph. Then, edges are drawn between nodes to show the flow of control. After constructing the CFG, the total number of nodes and edges are counted. Cyclomatic Complexity is then calculated using standard formulas. Finally, the number of independent paths is identified, and test cases are designed to cover all paths, ensuring complete path coverage.

About the Node Diagram:
The node diagram, also known as the Control Flow Graph (CFG), is a visual representation of a program that illustrates the flow of execution. In this diagram, each node represents a block of code or statement, while edges indicate the direction of control flow between these blocks. Decision nodes represent branching in the program, such as conditional statements or loops. The CFG helps in visualizing all possible execution paths and is useful for analyzing program complexity and designing effective test cases.


Virtual Labs | Computer Science
Virtual Labs
se-iitkgp.vlabs.ac.in/exp/metrics-structural-complexity/cfg.html
Estimation of Test Coverage Metrics and Structural Complexity
Rate Me
Report a Bug
Show Control Flow Graph
Control Flow Graph of the program
ENTRY
#2
n = 10;
sum = 0;
i = 1;
#4
if (i <= n)
#3
sum = sum +i;
i = i + 1;
#5
printf ("Sum of first %d natural number is: %d\n", n, sum);
#6
return 0;
EXIT
Determination of McCabe's Cyclomatic Complexity
No. of Nodes (N) 7
No. of Edges (E) 7
Cyclomatic Complexity V(G) 2
Check
Click to View Solution


Virtual Labs | Computer Science
Virtual Labs
se-iitkgp.vlabs.ac.in/exp/metrics-structural-complexity/cfg.html
Estimation of Test Coverage Metrics and Structural Complexity
Rate Me
Report a Bug
Show Control Flow Graph
Control Flow Graph of the program
ENTRY
#2
nr = 5;
nc = 10;
i = 0;
#7
if (i <= nr)
#3
j = 0;
#8
D.2061 = 0;
#5
if (j <= nc)
#6
i = i + 1;
#4
i.0 = i;
j.1 = j;
a[i.0][j.1] = 0;
j = j + 1;
#9
return 0;
EXIT
Determination of McCabe's cyclomatic complexity
No. of Nodes (N) 10
No. of Edges (E) 11
Cyclomatic Complexity V(G) 3
Check



Working of Nodes:

The working of nodes in a control flow graph involves understanding how each statement is connected and contributes to the execution of the program. The complexity of the graph is measured using Cyclomatic Complexity. The formulas used are:

$$V(G)=E-N+2$$

Where:

V(G)= Cyclomatic Complexity

E= Number of edges

N= Number of nodes

Another equivalent formula is:

$$V(G)=P+1$$

Where:

P= Number of predicate (decision) nodes

These formulas help in determining the number of independent paths in the program. Each independent path represents a unique execution flow that must be tested. Nodes work together to form these paths, ensuring that all logical conditions and loops are properly verified.

Result:

From the constructed control flow graph, the number of nodes and edges are identified, and Cyclomatic Complexity is calculated. For example, if the number of nodes is 10 and edges is 11,

then using the formula:
V(G)=11-10+2=3.
This indicates that there are 3 independent paths in the program. Therefore, at least 3 test cases are required to achieve complete path coverage. The results show that the program has moderate complexity and is manageable for testing.

Conclusion:
The experiment successfully shows how to estimate test coverage metrics and analyze structural complexity using a control flow graph. By calculating Cyclomatic Complexity, we can determine the minimum number of test cases required for effective testing. This approach improves software quality by ensuring that all independent paths are tested. It also helps developers understand program structure, reduce errors, and improve code design. Thus, CFG and Cyclomatic Complexity are important tools in software testing and quality assurance.